



FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

COMPUTER SCIENCE DEPARTMENT

CS0011

(Mobile Programming)

ACTIVITY

1

Logical Reasoning Code Analysis Debugging
Implementation

Name:	Dawn Andrei Pamesa
Professor:	Ma'am Jyra Rivera
Date Performed:	September 19, 2026
Date Submitted:	September 19, 2026

I. OBJECTIVES

This assessment measures whether you can reason about Kotlin programs, detect defects, justify control-flow choices, and build a small working solution using the concepts from the lectures.

Time	Total	Mode
90 minutes – 120 minutes	100 points	Individual

II. BACKGROUND INFORMATION

Kotlin is a modern programming language developed by JetBrains and widely used for Android application development. It provides concise syntax, Java interoperability, null-safety features, and support for object-oriented programming. The lectures introduce Kotlin fundamentals, including functions, variables declared with `val` and `var`, data types, string manipulation, nullable variables, conditional statements, when expressions, and `for`, `while`, and `do-while` loops. They also explain how to create, configure, and run Kotlin programs using Android Studio and a Kotlin JVM library module.

This technical assessment measures more than the students' ability to remember Kotlin syntax. It requires them to trace program execution, predict outputs, identify compilation and runtime problems, correct faulty code, compare possible solutions, and justify their programming decisions. These activities exercise logical thinking by requiring students to follow the sequence of program operations and critical thinking by asking them to evaluate whether a solution is correct, efficient, safe, and appropriate for a given requirement.

The practical component integrates several lecture concepts into one working Kotlin program. Students must select suitable variable declarations, handle nullable information safely, apply appropriate conditional and looping structures, calculate results, and verify that the program's output matches its requirements. The Android Studio troubleshooting questions also assess whether students can distinguish a code defect from a project or module configuration problem and evaluate suggested fixes using evidence rather than accepting them without verification.

III. LABORATORY INSTRUCTIONS

1. Show your reasoning. A correct answer without an explanation may receive only partial credit.
2. Assume standard Kotlin syntax and console output. Code does not need semicolons.
3. When correcting code, change only what is necessary and explain why the change works.
4. Do not use generative AI during Parts A to C. If tools are allowed for Parts D or E, you remain responsible for verifying every suggestion.

Part A Trace and Explain

For each item, state the exact output or result and explain the decisions made by the program.

A1: Trace the program. State the output and explain why score can change but bonus cannot be reassigned. [5 points]

Answer/Output here

- Bonus cannot be changed while score can be because score is labeled as val. Which makes it immutable.

```
fun main() {  
    var score = 72  
    val bonus = 8  
    score = score + bonus  
  
    if (score >= 75) {  
        println("PASS: $score")  
    } else {  
        println("RETRY: $score")  
    }  
}
```

A2: Trace every loop iteration. Give the final value of total and show how each number changes it. [5 points]

Answer/Output here

- The loop starts with total at 0 and iterates through numbers 1 to 5, subtracting 1 whenever n is odd and adding n whenever it is even. Across the five iterations, total changes sequentially from 0 to -1 (at n=1), 1 (at n=2), 0 (at n=3), 4 (at n=4), and finally 3 (at n=5). Consequently, the loop finishes and prints the final value of 3.

```
fun main() {  
    var total = 0  
    for (n in 1..5) {  
        if (n % 2 == 0) {  
            total = total + n  
        } else {  
            total = total - 1  
        }  
    }  
    println(total)  
}
```

A3: Predict the complete output. Explain why one loop runs at least once and the other loop does not run. [5 points]

Answer/Output here

- The do-while loop executes its body first before evaluating the condition, which allows it to print D3 and increment number to 4 even though number < 3 is false. In contrast, the standard while loop checks the condition before executing, so it skips entirely because number is already 4 (not less than 3). Therefore, only the do-while loop runs once, while the while loop does not run at all.

```

fun main() {
    var number = 3

    do {
        print("D$number ")
        number++
    } while (number < 3)

    while (number < 3) {
        print("W$number ")
        number++
    }
}

```

A4: State the two output lines. Then explain what must change in the declaration if course may legitimately contain null. [5 points]

Answer/Output here

- The two lines of output are 18 and Mobile, representing the total character length of the string (including the space) and the substring from index 0 up to index 6, respectively. To allow course to legitimately contain null, the declaration must include an explicit nullable type annotation with a question mark, changing it to `val course: String? = "Mobile Programming"` (or `var course: String? = "Mobile Programming"` if it needs to be reassigned later).

```

fun main() {
    val course = "Mobile Programming"
    println(course.length)
    println(course.substring(0, 6))
}

```

Part B Debug and Improve

Each program contains a defect. Identify the defect, provide the smallest valid correction, and justify the correction.

B1: The program must increase attempts from 1 to 2. Why does it fail to compile, and what is the smallest correction? [8 points]

Answer/Output here

- The program fails to compile because `attempts` is declared using `val`, which makes it a read-only (immutable) variable that cannot be reassigned. The smallest correction is to change the

keyword `val` to `var` in the variable declaration (`var attempts = 1`), which makes the variable mutable and allows its value to be updated.

```
fun main() {  
    val attempts = 1  
    attempts = attempts + 1  
    println(attempts)  
}
```

B2: The name is not yet known. Correct the null-safety problem so the program prints Unknown when name is null and prints the character count otherwise. [8 points]

Answer/Output here

- To resolve the null-safety error, the variable's type must be made nullable by changing `String` to `String?`, and the print statement must use the safe-call operator (`?.`) combined with the Elvis operator (`?:`). Specifically, change the declaration to `var studentName: String? = null` and update the printing line to `println(studentName?.length ?: "Unknown")`. This ensures that if `studentName` is null, it falls back to printing "Unknown", and otherwise prints the string's character count.

```
fun main() {  
    var studentName: String = null  
    println(studentName.length)  
}
```

B3: The intended output is 1 to 5, once each. Explain the runtime problem and correct the loop without replacing it with a for loop. [9 points]

Answer/Output here

- The runtime problem is an infinite loop because `number` is never updated inside the loop body, causing the condition `number <= 5` to always evaluate to true and repeatedly print 1 indefinitely. To correct the loop, increment `number` by adding `number++` (or `number += 1`) inside the loop body

```
fun main() {  
    var number = 1  
    while (number <= 5) {  
        println(number)  
    }  
}
```

Part C Evaluate Design Decisions

Select or design the better solution. Credit depends on the quality of your justification.

C1: A student ID is assigned once and must never change, while loginAttempts increases after each failed login. Choose val or var for each variable and explain how your choices prevent mistakes. [5 points]

Answer/Output here

- The studentId should use val because it is assigned once and must never change, while loginAttempts should use var because it increases after each failed login. Declaring studentId as val prevents mistakes because the compiler rejects any accidental reassignment, so the error is caught at compile time instead of at runtime. Using var for loginAttempts also signals clearly that the value is meant to change.

C2: A menu accepts the exact choices 1, 2, 3, and 4, plus an invalid-choice response. Would you use a long if else if chain or when? Defend your choice and sketch the structure. [5 points]

Answer/Output here

- I would use a when expression instead of a long if else if chain. The menu compares one variable against four exact values, which is exactly what when is designed for, and it avoids repeating the variable name in every branch. The structure takes choice as its subject and gives each accepted value its own branch (1, 2, 3, and 4), with a final else branch that handles the invalid choice. It also stays easier to extend if a fifth option is added later.

C3: A program must display a prompt once before checking whether a password is valid, then repeat while it is invalid. Choose while or do while and explain the execution difference that matters. [5 points]

Answer/Output here

- I would use a do while loop. The prompt must appear once before the password is checked, and a do while loop runs its body first before evaluating the condition, so the prompt is always displayed at least once. In contrast, a while loop checks the condition first, so it could skip the prompt entirely. Therefore only do while meets the requirement of prompting before validating.

C4: A grade of 95 is incorrectly labeled Passed by the code below. Explain the logical error and rewrite the decision so every grade receives the most specific correct label. [5 points]

Answer/Output here

- The logical error is the order of the conditions. Because grade >= 75 is tested first, any grade of 75 or above matches that branch immediately, so a grade of 95 is labeled Passed and the branches for 90 and 80 are never reached. To correct it, the conditions must run from the most specific to the least specific: grade >= 90 prints Excellent, then grade >= 80 prints Very Good, then grade >= 75 prints Passed, and the else branch prints Failed.

```
if (grade >= 75) {  
    println("Passed")  
} else if (grade >= 90) {  
    println("Excellent")  
} else if (grade >= 80) {  
    println("Very Good")  
} else {
```

```
println("Failed")
}
```

Part D Build and Defend a Solution

Weekly Study Tracker

Create a complete Kotlin console program that analyzes five days of study time. Use only concepts presented in the lectures.

- Declare `studentName` as a nullable String. If it is null, display Student: Unknown; otherwise display the name.
- Declare five Int values named `day1` through `day5`. Use these test values: 2, 1, 3, 0, and 4.
- Use a for loop over days 1 through 5. Inside the loop, use `when` to select the correct hours variable for the current day.
- For each day, use `if else` to print Target met when hours are at least 2; otherwise print Below target.
- Use a mutable `totalHours` variable and update it inside the loop.
- After the loop, print the total. Print Consistent if the total is at least 10, Developing if it is at least 5, and Needs improvement otherwise.

D1: Write the complete program. Your code must compile and meet every requirement. [15 points]

```
Answer/Output here
fun main() {
    val studentName: String? = null

    if (studentName == null) {
        println("Student: Unknown")
    } else {
        println("Student: $studentName")
    }

    val day1 = 2
    val day2 = 1
    val day3 = 3
    val day4 = 0
    val day5 = 4

    var totalHours = 0

    for (day in 1..5) {
        val hours = when (day) {
            1 -> day1
            2 -> day2
            3 -> day3
            4 -> day4
            else -> day5
        }

        totalHours = totalHours + hours

        if (hours >= 2) {
            println("Day $day: Target met")
        } else {
            println("Day $day: Below target")
        }
    }
}
```

```

    }
}

println("Total hours: $totalHours")

if (totalHours >= 10) {
    println("Consistent")
} else if (totalHours >= 5) {
    println("Developing")
} else {
    println("Needs improvement")
}
}
}

```

D2: For the required test values, write the expected output in order. Include the student line, five daily results, the total, and the final classification. [6 points]

Answer/Output here

- Student: Unknown
Day 1: Target met
Day 2: Below target
Day 3: Target met
Day 4: Below target
Day 5: Target met
Total hours: 10
Consistent
- The five test values total $2 + 1 + 3 + 0 + 4 = 10$, so the program prints Consistent because 10 is at least 10.

D3: Explain two design decisions in your solution. At least one explanation must discuss why a variable is val or var, nullable or non-nullable, or why a particular control structure fits the task. [4 points]

Answer/Output here

- The first decision is that totalHours is declared with var while day1 through day5 are declared with val. The daily values are fixed test data that must never change, but totalHours accumulates a new value on every iteration, so only it needs to be mutable. The second decision is using when inside the for loop to pick the correct hours variable. The loop counter is a single value compared against five discrete cases, which is what when handles most cleanly, so the selection stays in one place instead of a long if else if chain.

Part E Android Studio Reasoning

Use the Android Studio workflow from the lecture. Focus on evidence-based troubleshooting, not trial-and-error clicking.

E1: Put these actions in a defensible order and explain the two dependencies that control the order. [5 points]

Answer/Output here

- The order is: create a new project and activity, then choose Kotlin as the project language and finish project setup, then create Main.kt in the intended Kotlin source location and write fun main(), then run Main.kt and inspect the output, and finally, if a standalone main function

cannot run in the app module, create or select a Kotlin JVM library module, move Main.kt into it, and run again.

- The first dependency is that the project and its language must exist before Main.kt can be created, because the language choice determines the source location the file goes into. The second dependency is that running the program must come before moving it to a Kotlin JVM library module, since the move is only justified once the output shows the app module cannot run a standalone main.

- Run Main.kt and inspect the output.
- Choose Kotlin as the project language and finish project setup.
- Create a new project and activity.
- Create Main.kt in the intended Kotlin source location and write fun main().
- If a standalone main function cannot run in the app module, create or select a Kotlin JVM library module, move Main.kt into it, and run again.

E2: Android Studio offers Fix with AI after an error. Describe a four-step verification process that distinguishes a code defect from a project or module configuration defect before accepting the suggestion. [5 points]

Answer/Output here

- First, read the full error message and check its source, because an error naming a source file, line, and symbol points to a code defect while a Gradle sync or unresolved module error points to a configuration defect. Then reproduce the same code in a Kotlin JVM library module or a scratch file, because if it compiles there the problem is the configuration and not the code. Then check the module setup itself, confirming the file is in the correct source set, the module applies the Kotlin plugin, and the run configuration targets the right main class. Finally, read what the suggested fix actually changes and compare it against the cause found in the first three checks, then rebuild and confirm the original error is gone and no new one appeared.

Submission Check

- Every prediction includes reasoning.
- Every correction explains the original defect.
- The practical program includes all required variables and control structures.
- The expected output is consistent with the submitted code.

IV. RUBRICS

Part	Assessment criterion	Excellent	Proficient	Developing	Beginning	Maximum
A	Code tracing and explanation	Predicts all results accurately, shows a complete trace, and clearly explains the Kotlin behavior. 18–20	Mostly accurate with minor tracing or explanation errors. 14–17	Demonstrates partial understanding but misses important steps or outputs. 10–13	Gives incorrect or unsupported answers and shows little understanding of execution flow. 0–9	20
B	Debugging and improvement	Identifies the exact defect, supplies the smallest valid correction, and explains why it works. 23–25	Identifies and corrects most defects with generally sound explanations. 18–22	Finds some defects, but corrections or explanations are incomplete. 13–17	Cannot reliably identify defects or produces corrections that do not solve the problem. 0–12	25
C	Evaluation of design decisions	Selects appropriate variables and control structures and supports every choice with sound technical reasoning. 18–20	Makes appropriate choices with minor gaps in justification. 14–17	Some choices are suitable, but the reasoning is weak or inconsistent. 10–13	Choices do not match the requirements and are unsupported. 0–9	20
D	Solution development and defense	Produces a complete, correct, and logically organized Kotlin solution that meets all requirements; output and explanations are accurate. 23–25	Produces a mostly correct solution with minor syntax or logic errors. 18–22	Produces a partially working solution but misses important requirements. 13–17	Produces incomplete or nonfunctional code and cannot explain the design. 0–12	25

E	Android Studio reasoning	Correctly sequences the workflow, distinguishes code errors from configuration errors, and verifies proposed fixes using evidence. 9–10	Shows a generally correct workflow and diagnosis with minor omissions. 7–8	Understands some steps but relies on trial and error or gives incomplete verification. 5–6	Cannot determine the correct workflow or evaluate suggested fixes. 0–4	10
<i>TOTAL</i>						100